

FinSphere AI

End-to-End Technical Blueprint

Vector RAG

ML Analytics

ReAct Agent

FastAPI

A structured walkthrough of FinSphere AI's backend systems — covering core AI/ML concepts, file-by-file architecture, and step-by-step data flows in plain, understandable language.

What's Inside

01

Key Technical Concepts

02

Backend File Inventory

03

Step-by-Step Data Flow Diagrams

04

Frontend Architecture Overview

Table of Contents

0

1

0

2

0

2

B

0

3

0

4

Key Technical Concepts

Core AI/ML methods powering FinSphere AI

Backend File Inventory

Every file, its role, and its connections

LangChain Integration Layer

Orchestration, prompts, agents, and embeddings

Step-by-Step Data Flows

Document ingestion and conversational RAG

Frontend Architecture Overview

Next.js dashboard and visualisation layer

Part 1 — Key Technical Concepts

FinSphere AI is built on specialised AI and machine-learning techniques. Below is a plain-language explanation of every concept — what it does in general, and exactly how FinSphere AI puts it to work.

01

Vector RAG (Retrieval-Augmented Generation)

What it is: Traditional LLMs can hallucinate or lack access to your private files. RAG first *retrieves* relevant text snippets from your documents and then *injects* those snippets into the AI prompt — so every answer is grounded in real facts rather than assumptions.

How it's used: When you ask the Copilot a question, ChromaDB is queried for the most relevant paragraphs from your annual report, those are compiled into a secure prompt, and the result is sent to Gemini / OpenAI to produce a fact-based answer.

02

Vector Embeddings

What it is: Computers understand numbers, not raw text. An *embedding model* converts a sentence into a list of numbers (a vector) whose position in space mathematically reflects its meaning — semantically similar sentences cluster close together.

How it's used: The local model `sentence-transformers/all-MiniLM-L6-v2` is used. Every uploaded document is split into chunks, converted into 384-dimensional vectors, and stored in ChromaDB for fast retrieval.

03

Maximum Marginal Relevance (MMR)

What it is: A search algorithm that balances two goals simultaneously: returning chunks that are highly *relevant* to your query AND avoiding *redundant* chunks that repeat the same information.

How it's used: ChromaDB uses MMR to select up to 4 diverse text blocks as context for the AI, giving a broader overview of the document rather than repeating the same paragraph over and over.

04

Altman Z-Score (Solvency Metric)

What it is: A classic corporate-stability formula from 1968 that combines five financial ratios (measuring liquidity, leverage, profitability, and activity). Score below 1.81 = Distress Zone (high default risk); above 2.99 = Safe Zone.

How it's used: The ML engine runs these calculations dynamically on your balance-sheet inputs to instantly classify a company's default risk zone.

05

Isolation Forest (Anomaly Detection)

What it is: An unsupervised ML algorithm designed to identify outliers. Rather than modelling normal data, it *isolates* anomalies — since anomalies are few and structurally different, they separate from the cluster quickly.

How it's used: Scans combinations of operating margin and financial leverage, flagging structural accounting errors that fall far outside the expected historical cluster.

06

Ridge Regression (Revenue Forecasting)

What it is: A linear regression model with an L2 regularisation penalty that shrinks large coefficients, preventing the model from overfitting to noise in small datasets.

How it's used: Models historical quarterly revenue sequences (Q1-Q4) to forecast the next quarter's revenue trajectory reliably.

07

ReAct Autonomous Agent (Reasoning + Acting)

What it is: A paradigm where an AI loops through: **Thought** (decide what to do next) → **Action** (invoke a specific tool) → **Observation** (analyse the result) — repeating until the complex query is fully resolved.

How it's used: The Analyst Copilot uses ReAct to decide whether to query the document vault, run solvency ratios, or detect anomalies — then combines all findings into a structured SWOT analysis.

Part 2 — Backend File Inventory & Connections

Every backend file is listed below, grouped by functional layer. Each card shows the file's role and which other files it interacts with.

Core Config & Bootstrapping

main.py

Role: The application entry point. Initialises the FastAPI app, injects CORS middleware so the Next.js frontend can communicate, and registers all route controllers under /api/v1.

Connects to: auth.py, documents.py, ml.py, portfolio.py, agents.py, rag.py

core/config.py

Role: Reads all environment variables from .env. Handles secure file-fallback operations for private session signature keys stored in jwt_secret.txt.

Connects to: Used by virtually every service for file paths, DB directories, and API keys

api/deps.py

Role: Provides FastAPI dependency-injection helpers. Decodes incoming HttpOnly JWT cookies, verifies the logged-in user, and blocks unauthorised requests before they reach business logic.

Connects to: Used as a security boundary by almost all router files

Security & Relational Database

core/security.py

Role: Handles all cryptography. Uses bcrypt to hash user passwords at signup and verify them at login. Generates signed JWT session tokens.

Connects to: api/v1/auth.py

database/connection.py

Role: Manages SQL connection pools via SQLAlchemy with a 30-second queue timeout to prevent SQLite deadlocks under concurrent load.

Connects to: api/deps.py — yields database sessions to active requests

database/models.py

Role: Declares the relational schema: User, Document, and PortfolioHistory tables using declarative SQLAlchemy classes.

Connects to: Referenced by routers to query and write relational rows

Document Ingestion & Vector RAG Pipeline

services/ingestion.py

Role: The parser layer. Extracts text from PDF annual reports (PyPDF), plain-text call transcriptions, and CSV tables (Pandas). Splits content into semantically connected chunks (~1,000 characters with 200-character overlap).

Connects to: api/v1/documents.py → rag/vector_store.py

rag/vector_store.py

Role: Manages the local vector index. Instantiates ChromaDB and configures SentenceTransformers to generate 384-dimensional embeddings from each text chunk.

Connects to: ingestion.py (write vectors) and pipeline.py (search vectors)

rag/pipeline.py

Role: Executes MMR vector searches isolated by user_id (preventing cross-tenant data leaks). With API key: calls Gemini/OpenAI via secure REST. Offline: smart text parser strips test boilerplate and returns a clean analyst summary.

Connects to: api/v1/rag.py and agents/tools.py

Machine Learning & Quantitative Analytics

ml/train.py

Role: Training pipeline. Prepares and fits Ridge models for revenue forecasting, Isolation Forests for anomaly detection, and Random Forests for bankruptcy classification. Saves trained models as binary .joblib files.

Connects to: Exports to backend/app/ml/models/

ml/prediction.py

Role: Inference engine. Loads the trained .joblib models to evaluate incoming balance-sheet parameters, predict operating margins, and flag anomalies in real time.

Connects to: api/v1/ml.py and agents/tools.py

services/sentiment.py

Role: Sentiment classifier. Splits earnings-call transcript blocks, uses TF-IDF vectorisers to classify paragraphs as positive or negative, and extracts executive commitments for reporting.

Connects to: api/v1/ml.py — called when a transcript is submitted

services/portfolio_engine.py

Role: Portfolio allocator. Evaluates user risk profiles and target industries, returning mathematically diversified asset distributions.

Connects to: api/v1/portfolio.py

Autonomous ReAct Agent Core

agents/analyst_agent.py

Role: Autonomous coordinator. Integrates with Gemini or OpenAI models, prompting them to execute multi-step financial analysis by invoking specialised Python helper functions (tools). Bound to /api/v1/analyst/run-task.

Connects to: agents/tools.py

agents/tools.py

Role: The agent's hands. Exposes core system capabilities as callable tools: run_bankruptcy_risk_calculator, search_document_vault, run_revenue_forecaster.

Connects to: ml/prediction.py and rag/pipeline.py

Part 2B — LangChain Integration Layer

LangChain is used extensively throughout the FinSphere AI backend. It acts as the primary orchestration framework — handling RAG integrations, conversational memory, vector database wrappers, and autonomous agent loop routing. Below is a precise breakdown of every place LangChain is used and what it does there.

vector_store.py	pipeline.py	analyst_agent.py
Embeddings & Vector DB	Prompts & LLM Clients	ReAct Agent Core

vector_store.py — Vector Database & Embedding Client

This file is the bridge between FinSphere AI's document pipeline and ChromaDB. LangChain is used here to wrap both the vector store and the embedding models, providing a clean, unified API regardless of whether the system runs locally or against a cloud OpenAI endpoint.

Vector Store Wrapper `from langchain_community.vectorstores import Chroma`

Wraps ChromaDB with LangChain's Chroma interface, enabling clean initialisation, MMR-based querying, and targeted deletion of document vectors — all with LangChain's standardised retriever API.

Local Sentence Embeddings `from langchain_community.embeddings import HuggingFaceEmbeddings`

Runs the local sentence-transformers/all-MiniLM-L6-v2 model via LangChain's HuggingFace wrapper. Generates 384-dimensional embeddings on CPU/GPU without any external API calls.

OpenAI Embeddings (Cloud Fallback) `from langchain_openai import OpenAIEmbeddings`

When an OpenAI API key is configured, this client is activated to fetch higher-dimensional cloud embeddings dynamically — providing a seamless local-to-cloud upgrade path.

pipeline.py — Conversational RAG Pipeline

The RAG pipeline uses LangChain to enforce strict hallucination-prevention rules and to manage the LLM client configuration. LangChain's prompt templates act as guardrails that require the model to ground every claim in retrieved document context.

Prompt Templates & Anti-Hallucination Guards `from langchain.prompts import PromptTemplate`

PromptTemplate is used to design structured prompts with strict guidelines — enforcing quantitative citations from retrieved document chunks and explicitly blocking the AI from generating ungrounded financial claims.

LLM Client (High-Precision Mode)

```
from langchain_openai import ChatOpenAI
```

Initialises the OpenAI chat model with precision-tuned parameters (e.g. temperature=0.0) via LangChain's interface, ensuring deterministic, fact-grounded responses for financial analysis use cases.

analyst_agent.py — Autonomous ReAct Agent

The Analyst Copilot's autonomous reasoning loop is entirely orchestrated by LangChain. The AgentExecutor manages the Thought → Action → Observation cycle, while ChatPromptTemplate structures the system persona and preserves multi-turn conversation history.

Agent Coordinator & Tool Router

```
from langchain.agents import AgentExecutor, create_openai_tools_agent
```

Creates the autonomous ReAct agent and binds it to FinSphere AI's custom Python analytical tools (bankruptcy calculator, document vault search, revenue forecaster). AgentExecutor manages the full reasoning loop and coordinates inputs and outputs across tool calls.

Structured Chat Prompts & Memory

```
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
```

ChatPromptTemplate defines the agent's system persona and instructions. MessagesPlaceholder enables clean injection of structured multi-turn chat histories into each request, so the agent retains full conversation context across complex multi-step analysis sessions.

LangChain Usage Summary

File	LangChain Component	Purpose
vector_store.py	langchain_community Chroma	Vector store wrapper for ChromaDB
vector_store.py	langchain_community HuggingFaceEmbeddings	Local CPU/GPU embedding model
vector_store.py	langchain_openai OpenAIEmbeddings	Cloud embedding fallback
pipeline.py	langchain PromptTemplate	Anti-hallucination prompt guards
pipeline.py	langchain_openai ChatOpenAI	Precision LLM client (temp=0.0)
analyst_agent.py	langchain AgentExecutor	ReAct loop coordinator
analyst_agent.py	langchain create_openai_tools_agent	Tool-binding agent factory
analyst_agent.py	langchain ChatPromptTemplate	System persona & instructions
analyst_agent.py	langchain MessagesPlaceholder	Multi-turn memory injection

Part 3 — Step-by-Step Data Flows

Flow 1 — Document Ingestion

1	HTTP Upload	The Next.js client sends a multipart/form-data POST request containing a PDF, CSV, or TXT file to the backend.
2	Auth Check	api/deps.py intercepts the request, validates the HttpOnly JWT cookie, and extracts the authenticated user_id.
3	Parsing	The file is forwarded to services/ingestion.py which extracts the full text content using the appropriate parser.
4	Chunking	Text is split into semantically connected blocks of approximately 1,000 characters with a 200-character overlap to preserve context at boundaries.
5	Vector Processing	rag/vector_store.py runs each chunk through SentenceTransformers to generate 384-dimensional embedding vectors.
6	Indexing	Vectors and metadata (user_id, filename, doc_type) are persisted in ChromaDB, ready for MMR retrieval.

Flow 2 — Conversational RAG Query

1	Query Submitted	The client sends a natural-language question, e.g. 'Summarise earnings-call concerns', via a POST request.
2	Vector Retrieval	rag/pipeline.py queries ChromaDB using MMR, filtering results strictly by user_id to guarantee full tenant isolation.
3	Context Compilation	The top-4 diverse, relevant chunks are assembled into a grounded prompt template alongside the user's question.
4	AI Generation (Online)	If GEMINI_API_KEY is configured: a secure REST POST is sent to the Google Gemini API with the compiled context + question.
5	Offline Fallback	Without an API key: the engine parses raw text chunks locally, strips test-suite boilerplate lines, and formats a clean structured analyst summary.
6	Response Returned	The generated answer and the names of the source documents consulted are returned to the client for display.

Part 4 — Frontend Architecture Overview

The frontend is a single-page Next.js dashboard that communicates asynchronously with the FastAPI backend, passing secure HttpOnly JWT credentials on every request.

Theme & Aesthetics	Dark-mode glassmorphic design (globals.css) with CSS blur filters, animated neon shadows, and a dot-grid background — giving the app a premium financial-terminal look and feel.
Routing	Next.js App Router drives the main views: /dashboard, /dashboard/upload, /dashboard/chat, /dashboard/analytics, /dashboard/portfolio, and /dashboard/earnings.
Data Visualisation	Recharts renders gradient-filled area charts for revenue trends and dynamic ring charts for portfolio allocation — all fed with live data from the backend APIs.
API Communication	All client pages communicate with the FastAPI backend through services/api.ts, automatically forwarding secure HttpOnly JWT cookies to authenticate each request and render fresh financial metrics without page reloads.

FinSphere AI — End-to-End Backend Architecture Report

This document covers all backend layers: AI/ML concepts, file connections, data flows, and the frontend interface. For implementation details, refer to the inline source files referenced throughout.